PYTHON - SE - BACKEND

SOFTWARE ENGINEERING – MODULE 3

Introduction to OOPS Programming

SESSION / ASSIGNMENT – 5

TASKS :

1. **Create a Java class called PaymentProcessor with two overloaded methods processPayment(): one that takes only an amount, and one that takes amount and a coupon code. Print which version is called and the final amount in each case.**

➔

```cpp
#include <iostream>
#include <string>
using namespace std;

class PaymentProcessor {
public:

    // Method with only amount
    void processPayment(double amount) {
        cout << "processPayment(amount) called" << endl;
        cout << "Final Amount: " << amount << endl;
    }

    // Overloaded method with amount and coupon code
    void processPayment(double amount, string couponCode) {
        cout << "processPayment(amount, couponCode) called" << endl;

        double discount = 0;

        if (couponCode == "SAVE10") {
            discount = amount * 0.10;
        }

        double finalAmount = amount - discount;

        cout << "Coupon Code: " << couponCode << endl;
        cout << "Final Amount: " << finalAmount << endl;
    }
};
```

```cpp
int main() {

    PaymentProcessor payment;

    // Calling method with only amount
    payment.processPayment(1000);

    cout << endl;

    // Calling overloaded method with amount and coupon
    payment.processPayment(1000, "SAVE10");

    return 0;
}
```

OUTPUT:

```
processPayment(amount) called
Final Amount: 1000

processPayment(amount, couponCode) called
Coupon Code: SAVE10
Final Amount: 900
```

2. **Build two classes, InstagramUploader and YouTubeUploader, each with a method uploadContent(). Both should extend a base class SocialMediaUploader and override uploadContent() to print a message showing how uploading works differently for Instagram and YouTube.**

➔

```cpp
#include <iostream>
#include <string>
using namespace std;

class SocialMediaUploader {
public:
    virtual void uploadContent() {
        cout << "Uploading content to social media..." << endl;
    }
};

class InstagramUploader : public SocialMediaUploader {
public:
    void uploadContent() override {
        cout << "Uploading photo/video to Instagram with filters and hashtags." << endl;
    }
};

class YouTubeUploader : public SocialMediaUploader {
public:
    void uploadContent() override {
        cout << "Uploading video to YouTube with title, description, and thumbnail." << endl;
    }
};

int main() {

    InstagramUploader instagram;
    YouTubeUploader youtube;

    instagram.uploadContent();

    youtube.uploadContent();

    return 0;
}
```

OUTPUT:

```
Uploading photo/video to Instagram with filters and hashtags.
Uploading video to YouTube with title, description, and thumbnail.
```

3. **Write a function in Java or Python that simulates a Flipkart-style search: overload a method searchProduct() to allow searching by product name or by product name and category. Demonstrate both usages with sample data.**

➔

```cpp
#include <iostream>
#include <string>
using namespace std;

class FlipkartSearch {
public:

    // Search by product name
    void searchProduct(string productName) {
        cout << "Searching for product: " << productName << endl;

        cout << "Products found:" << endl;
        cout << "- " << productName << " available" << endl;
    }

    // Search by product name and category
    void searchProduct(string productName, string category) {
        cout << "Searching for product: " << productName << endl;
        cout << "Category: " << category << endl;

        cout << "Products found:" << endl;
        cout << "- " << productName << " in " << category << " category" << endl;
    }
};

int main() {

    FlipkartSearch search;

    // Search only by product name
    search.searchProduct("Laptop");

    cout << endl;

    // Search by product name and category
    search.searchProduct("Laptop", "Electronics");

    return 0;
}
```

OUTPUT:

```
Searching for product: Laptop
Products found:
- Laptop available

Searching for product: Laptop
Category: Electronics
Products found:
- Laptop in Electronics category
```

4. **Given this code: class MusicPlayer { void play(String song) { System.out.println("Playing: " + song); } } class SpotifyPlayer extends MusicPlayer { void play(String song) { System.out.println("Streaming on Spotify: " + song); } } — Create an object of type MusicPlayer but assign it a SpotifyPlayer instance, then call play(). Explain the output.**
   Hint: This tests runtime polymorphism (overriding) and dynamic method dispatch.

➔

```cpp
#include <iostream>
#include <string>
using namespace std;

class MusicPlayer {
public:
    virtual void play(string song) {
        cout << "Playing: " << song << endl;
    }

    virtual ~MusicPlayer() {
    }
};

class SpotifyPlayer : public MusicPlayer {
public:
    void play(string song) override {
        cout << "Streaming on Spotify: " << song << endl;
    }
};

int main() {

    // Base class pointer pointing to derived class object
    MusicPlayer* player = new SpotifyPlayer();

    player->play("Blinding Lights");

    delete player;
```

```
    return 0;
}
```

OUTPUT:

```
Streaming on Spotify: Blinding Lights
```